Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Санкт-Петербургский политехнический университет Петра Великого»
—
Институт компьютерных наук и кибербезопасности
Высшая школа кибербезопасности

ЛАБОРАТОРНАЯ РАБОТА № 3

«Работа с таймерами-счётчиками»

по дисциплине «Аппаратные средства вычислительной техники»

Выполнил
студент гр. 5151003/40001 Корпусов Д.Н

<подпись>

Руководитель Макаров А.С

<подпись>

Санкт-Петербург
2025

Содержание

Цель работы 3
Ход работы 3-15
1. Теоретическая часть 7-13
2. Практическая часть 6-15
2.1 Схема установки 7-8
2.2 Описание алгоритма 8-11
Ответы на контрольные вопросы 12-13
Вывод 14
Листинг «.asm-файл» 15-44

ЦЕЛЬ РАБОТЫ

Получение практических навыков по работе с таймерами-счётчиками и применению механизма прерываний.

ХОД РАБОТЫ

1. Теоретическая часть

Микроконтроллеры AVR, в том числе ATmega32, имеют встроенные аппаратные таймеры-счётчики. Таймер-счётчик представляет собой специальный регистр, значение которого автоматически увеличивается под действием тактовых импульсов. В отличие от программных задержек, аппаратный таймер работает независимо от выполнения основной программы, поэтому его удобно использовать для точного отсчёта времени.

Таймер может работать в двух основных вариантах: как таймер и как счётчик. В режиме таймера он считает импульсы от внутренней тактовой частоты микроконтроллера. В режиме счётчика он может считать внешние импульсы, поступающие на специальный вывод микроконтроллера. Таким образом, таймер используется для формирования временных интервалов, а счётчик – для подсчёта внешних событий.

Частота работы таймера может быть уменьшена с помощью предделителя. Предделитель делит основную частоту микроконтроллера на заданное число, например на 8, 64, 256 или 1024. Это нужно, чтобы таймер не переполнялся слишком быстро и чтобы можно было получать удобные временные интервалы.

Для управления таймером используются специальные регистры. В ATmega32 для таймера Timer0 применяются:

1. TCCR0 – регистр управления таймером.
2. TCNT0 – счётный регистр таймера.
3. OCR0 – регистр сравнения.
4. TIMSK – регистр разрешения прерываний.
5. TIFR – регистр флагов событий таймера.

TCCR0 определяет источник тактирования таймера, коэффициент предделителя, режим работы таймера-счётчика T0 и поведение вывода OC0.

Биты CS02, CS01, CS00 (Clock Select) – определяют источник тактовой частоты для таймера T0 и задают коэффициент предделителя.

7	6	5	4	3	2	1	0
FOC0	WGM00	COM01	COM00	WGM01	CS02	CS01	CS00
W	R/W	R/W	R/W	R/W	R/W	R/W	R/W

Рисунок 1 – Регистр TCCR0 (Timer/Counter Control Register)

Биты WGM01, WGM00 (Wave Generator Mode) – определяют режим работы таймера-счётчика T0. Всего их может быть четыре – нормальный режим (Normal), сброс таймера при совпадении (CTC), и два режима широтно-импульсной модуляции (FastPWM и Phase Correct PWM).

WGM01	WGM00	Режим работы таймера-счётчика
0	0	Normal
0	1	PWM, Phase correct
1	0	CTC
1	1	Fast PWM

Счётный регистр TCNT0 хранит текущее значение таймера. При каждом тактовом импульсе его значение изменяется. Регистр OCR0 используется для сравнения: если значение TCNT0 становится равно значению OCR0, возникает событие совпадения.

Один из режимов работы таймера – режим CTC, то есть Clear Timer on Compare Match. В этом режиме таймер считает от нуля до значения, записанного в OCR0. Когда происходит совпадение TCNT0 = OCR0, таймер сбрасывается обратно в ноль. Это позволяет получать периодические события через строго заданные промежутки времени.

Важным механизмом микроконтроллеров являются прерывания. Прерывание – это временная остановка выполнения основной программы для обработки некоторого события. После выполнения обработчика прерывания

микроконтроллер возвращается к основной программе. Прерывания позволяют не опрашивать постоянно состояние таймера или кнопок, а реагировать на событие только тогда, когда оно произошло.

В лабораторной работе также используется внешний ввод с кнопок. Кнопки могут подключаться к выводам микроконтроллера и формировать на них логический ноль или логическую единицу. Для работы с портами ввода-вывода используются регистры:

DDRx – задаёт направление выводов порта;

PORTx – задаёт выходное значение или включает подтяжку;

PINx – позволяет прочитать текущее состояние входов.

Если бит в DDRx равен 1, соответствующий вывод работает как выход. Если бит равен 0, вывод работает как вход. Поэтому для кнопок выводы настраиваются как входы, а для управления индикатором — как выходы.

Для отображения информации используется семисегментный индикатор. Одна цифра такого индикатора состоит из семи основных сегментов A–G и дополнительной точки DP. Чтобы вывести цифру, нужно включить определённый набор сегментов. Например, для цифры 1 включаются только два правых вертикальных сегмента, а для цифры 8 включаются все семь сегментов.

Если используется четырёхразрядный семисегментный индикатор, его часто подключают по схеме мультиплексирования. При мультиплексировании линии сегментов у всех разрядов общие, а каждый разряд имеет отдельную линию выбора. В каждый момент времени включается только один разряд, но переключение происходит очень быстро. Из-за инерционности зрения человек видит, будто все разряды горят одновременно.

Таким образом, микроконтроллер по очереди выбирает разряды индикатора и выводит для каждого свой набор сегментов. Такой способ позволяет уменьшить количество используемых выводов микроконтроллера.

Ещё один важный момент – активный уровень элементов схемы. Некоторые индикаторы или светодиоды могут быть активны по единице, а некоторые – по нулю. Если элемент активен по нулю, то логический 0 включает сегмент, а логическая 1 выключает его. Поэтому при работе с такими индикаторами код сегментов необходимо инвертировать.

Временные интервалы в работе таймера, обработке кнопок и мигании индикатора формируются не программными задержками, а с помощью периодических прерываний таймера. Такой подход является более правильным, потому что микроконтроллер не простаивает в циклах задержки, а может выполнять другие действия между прерываниями.

Таким образом, в данной лабораторной работе рассматриваются основные механизмы работы с таймерами-счётчиками, прерываниями, портами ввода-вывода и семисегментной индикацией. Эти механизмы позволяют организовать точный отсчёт времени, обработку кнопок и отображение информации без использования программных задержек.

2. Практическая часть

2.1 Схема установки

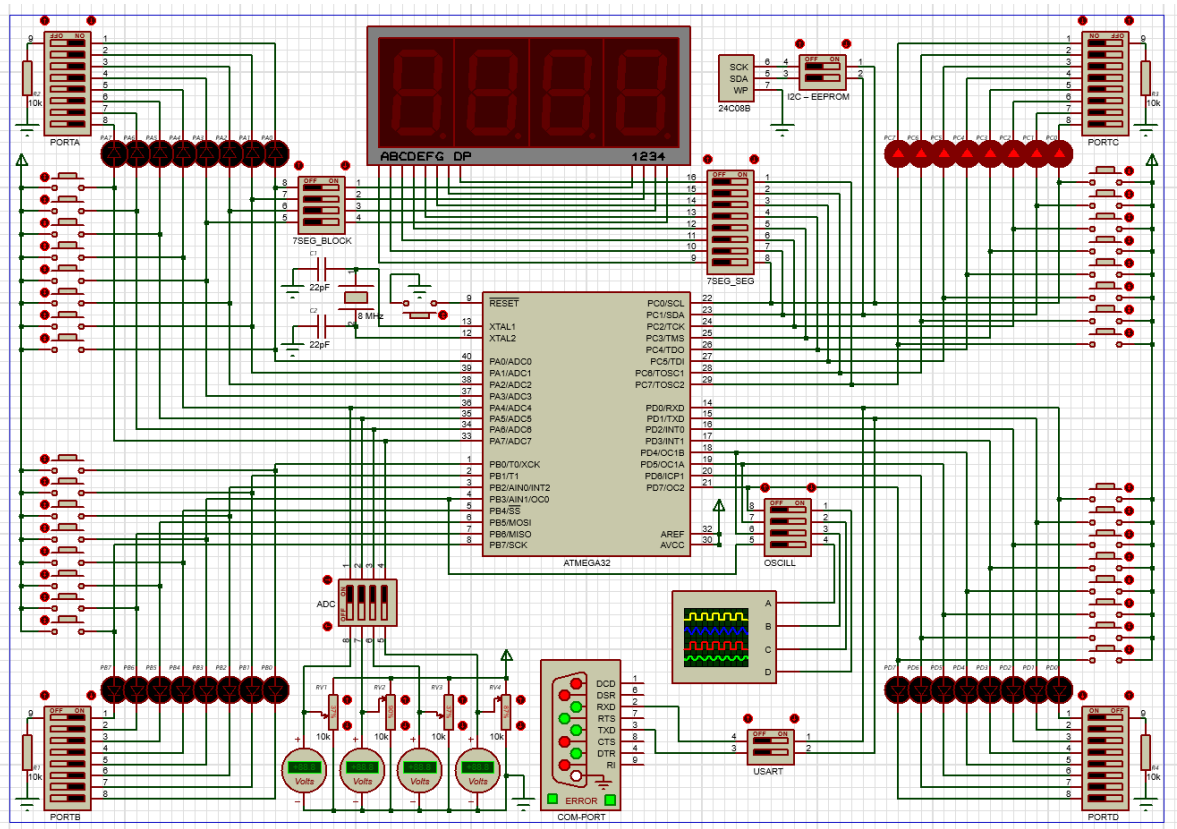


Рисунок 2 – Схема установки

В работе используется микроконтроллер ATmega32, четырёхразрядный семисегментный индикатор, кнопки управления и кварцевый резонатор 8 МГц.

Узел	Назначение
ATmega32	Выполнение программы таймера
Timer0	Формирование периодических прерываний
PORTC	Управление сегментами A, B, C, D, E, F, G, DP
PORTA	Выбор активного разряда индикатора
PD0	Увеличение значение таймера на 1 секунду
PD1	Уменьшение значение таймера на 1

	секунду
PD2 / INT0	Запуск, пауза и продолжение отсчёта
PD3 / INT1	Переключение режима настройки и режима работы
Кварц 8 МГц	Тактирование микроконтроллера

Сегменты индикатора подключены к линиям PC0..PC7. Выбор разрядов выполняется через PA0..PA3. Кнопки подключены к PD0..PD3 и при нажатии формируют логическую единицу.

2.2 Описание алгоритма

После включения микроконтроллер настраивает стек, порты ввода-вывода, внешние прерывания INT0, INT1 и таймер Timer0.

PORTC настраивается как выход для управления сегментами индикатора. PORTA настраивается как выход для выбора активного разряда. PORTD используется как вход для кнопок.

Таймер Timer0 работает в режиме CTC. Он вызывает прерывание каждые 0,25 мс. В каждом прерывании обновляется один разряд индикатора. Каждые четыре таких прерывания получается интервал 1 мс, который используется для обработки кнопок, обратного отсчёта и мигания.

В режиме настройки кнопки PD0 и PD1 изменяют установленное значение таймера. Краткое нажатие изменяет значение на одну секунду. Если кнопка удерживается более двух секунд, изменение выполняется каждые 0,2 с, а после четырёх секунд — каждые 0,1 с.

Кнопка PD2, подключённая к INT0, запускает, приостанавливает и возобновляет таймер. Кнопка PD3, подключённая к INT1, переключает режим настройки и режим работы.

Когда значение таймера достигает 00.00, отсчёт прекращается. После этого значение 00.00 мигает с частотой 2 Гц.

Сегменты индикатора подключены к линиям PC0..PC7. Выбор разрядов выполняется через PA0..PA3. Кнопки подключены к PD0..PD3 и при нажатии формируют логическую единицу.



Рисунок 3 – Блок схема алгоритма



Рисунок 4 – Блок схема алгоритма

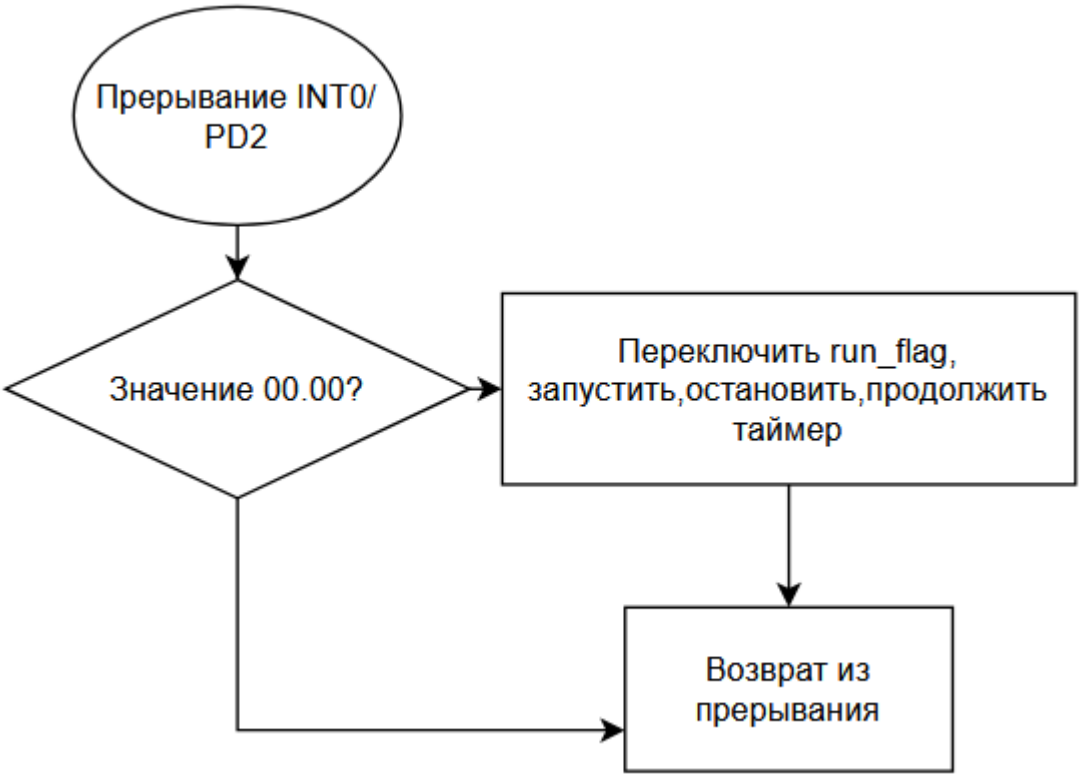


Рисунок 5 – Блок схема алгоритма

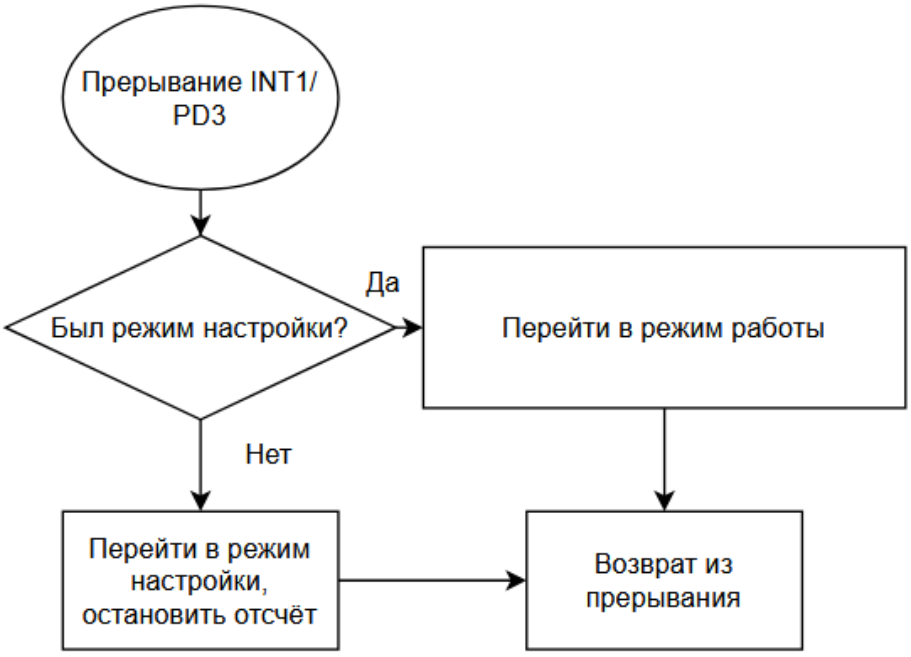


Рисунок 6 – Блок схема алгоритма

ОТВЕТЫ НА КОНТРОЛЬНЫЕ ВОПРОСЫ

1)Посредством каких регистров производится конфигурирование таймера-счётчика?

Конфигурирование таймера-счётчика выполняется с помощью специальных регистров управления. Для таймера Timer0 микроконтроллера ATmega32 основным регистром является TCCR0. В нём задаётся режим работы таймера, источник тактирования и коэффициент предделителя. Также используются регистры TCNT0, OCR0, TIMSK и TIFR. Регистр TCNT0 хранит текущее значение счётчика, OCR0 используется для сравнения, TIMSK разрешает прерывания таймера, а TIFR содержит флаги событий таймера.

2) Какие источники импульсов могут применяться для увеличения таймера-счётчика и для каких целей?

Для увеличения таймера-счётчика могут использоваться внутренние и внешние источники импульсов. Внутренним источником является тактовая частота микроконтроллера, обычно поданная через предделитель. Такой режим используется для отсчёта временных интервалов. Внешним источником могут быть импульсы, поступающие на специальный вход микроконтроллера. Такой режим используется для подсчёта внешних событий, например импульсов от датчика или другого устройства.

3) В каких режимах могут работать таймеры-счётчики?

Таймеры-счётчики могут работать в нескольких режимах. Основные режимы для Timer0 ATmega32: нормальный режим Normal, режим сброса при совпадении CTC, режим фазокорректной широтно-импульсной модуляции Phase Correct PWM и режим быстрой широтно-импульсной модуляции Fast PWM. В нормальном режиме таймер считает до переполнения, в режиме CTC сбрасывается при совпадении с регистром сравнения, а режимы PWM используются для формирования импульсных сигналов с заданной скважностью.

4) Как рассчитать начальное значение таймера-счётчика по заданному времени, которое должен отмерить таймер-счётчик до своего переполнения?

Для расчёта начального значения нужно определить, сколько тактов таймера должно пройти за заданное время. Сначала вычисляется частота таймера с учётом предделителя: частота микроконтроллера делится на коэффициент предделителя. Затем количество тактов находится как произведение этой частоты на заданное время. Для 8-разрядного таймера максимальное значение равно 256, поэтому начальное значение можно найти по формуле: TCNT0 = 256 - N, где N — количество тактов, которое должно пройти до переполнения. Если используется режим сравнения CTC, то вместо начального значения обычно рассчитывают значение регистра OCR0.

5) В чём состоит отличие работы таймера-счётчика в режиме таймера и в режиме счётчика?

В режиме таймера счётчик увеличивается от внутренней тактовой частоты микроконтроллера. Этот режим используется для измерения времени и формирования задержек или периодических событий. В режиме счётчика значение увеличивается от внешних импульсов, поступающих на специальный вывод микроконтроллера. Такой режим используется не для отсчёта времени, а для подсчёта количества внешних событий.

ВЫВОД

В ходе лабораторной работы были изучены принципы работы таймеров-счётчиков и прерываний микроконтроллера ATmega32. Была разработана программа таймера с настройкой времени, запуском, паузой и продолжением отсчёта. Временные интервалы формировались с помощью Timer0 в режиме CTC, без использования программных задержек. Также была реализована работа с кнопками и мультиплексный вывод значения таймера на четырёхразрядный семисегментный индикатор.

Листинг «.asm файл»

```
;
================================================================
=============
; Лабораторная 3, вариант 5. Таймер на ATmega32.
;
; Что делает программа:
;      1) Показывает время в формате MM.SS на 4-разрядном
семисегментном индикаторе.
;   2) В режиме настройки:
;        PD0 -> увеличить время на 1 секунду.
;        PD1 -> уменьшить время на 1 секунду.
;        Удержание PD0/PD1:
;          после 2 секунд изменение каждые 0.2 секунды;
;          после 4 секунд изменение каждые 0.1 секунды.
;   3) PD2 / INT0 -> запуск, пауза, продолжение таймера.
;   4) PD3 / INT1 -> переключение режима настройка/работа.
;    5) Когда таймер доходит до 00.00, он останавливается, а
00.00 мигает 2 Гц.
;
; Подключение в Proteus:
;   PC0..PC7 -> A,B,C,D,E,F,G,DP индикатора.
;   PA0..PA3 -> выбор разряда индикатора 1,2,3,4.
;   PD0      -> кнопка увеличения.
;   PD1      -> кнопка уменьшения.
;   PD2      -> кнопка запуска/паузы, внешнее прерывание INT0.
;   PD3      -> кнопка режима, внешнее прерывание INT1.
;
================================================================
=============

.include  "m32def.inc"                          ;  Подключаем  имена
регистров/битов ATmega32.
```

```
.def tmp    = r16                       ; tmp: главный временный
регистр для разных операций.
.def tmp2   = r17                       ; tmp2: второй временный
регистр, часто старший байт 16-битного числа.
.def digit = r18                        ; digit: номер текущего
разряда индикатора или временный регистр.
.def value = r19                    ; value: значение цифры, флаг
результата или период повторения.
.def pins  = r20                    ; pins: сюда читаем состояние
кнопок из PIND.
.def zero  = r1                         ; zero: регистр, который
держим равным 0.

.cseg                                   ; Далее начинается сегмент
программного кода.
.org 0x0000                         ; Адрес 0x0000 - точка старта
после сброса МК.
    rjmp RESET                      ; После reset сразу переходим
к метке RESET.
.org INT0addr                           ; Адрес вектора внешнего
прерывания INT0.
    rjmp INT0_ISR                   ; Если сработал INT0/PD2,
идём в обработчик INT0_ISR.
.org INT1addr                           ; Адрес вектора внешнего
прерывания INT1.
    rjmp INT1_ISR                   ; Если сработал INT1/PD3,
идём в обработчик INT1_ISR.
.org OC0addr                        ; Адрес вектора прерывания
Timer0 Compare Match.
    rjmp TIMER0_COMP_ISR            ; Если Timer0 досчитал до
OCR0, идём в обработчик таймера.

RESET:                                  ; Начальная настройка
микроконтроллера.
    ldi  tmp, high(RAMEND)          ; Загружаем старший байт
адреса конца SRAM.
```

```
    out SPH, tmp                        ; Записываем его в старший
байт указателя стека.
    ldi tmp, low(RAMEND)                ; Загружаем младший байт
адреса конца SRAM.
    out SPL, tmp                        ; Записываем его в младший
байт указателя стека.
    clr zero                            ; Обнуляем r1, дальше
используем zero как постоянный ноль.

    ; ---------------- Настройка портов индикатора -------------
---
    ldi tmp, 0x0F                       ; 00001111: PA0..PA3 будут
выходами.
    out DDRA, tmp                       ; Настраиваем PA0..PA3 как
выходы выбора разрядов.
    clr tmp                           ; tmp = 0.
    out PORTA, tmp                      ; Пока выключаем все разряды
индикатора.

    ldi tmp, 0xFF                      ; 11111111: все 8 линий PORTC
будут выходами.
    out DDRC, tmp                       ; Настраиваем PC0..PC7 как
выходы сегментов.
    clr tmp                           ; Для активной единицы 0x00
означает "все сегменты погасить".
    out PORTC, tmp                      ; Гасим все сегменты
индикатора.

    ; ---------------- Настройка кнопок ----------------
    clr tmp                           ; tmp = 0.
    out DDRD, tmp                       ; Весь PORTD делаем входом:
PD0..PD3 - кнопки.
    out PORTD, tmp                      ; Внутренние pull-up
отключены, потому что в схеме есть pull-down.
```

```
    ; ---------------- Начальные значения переменных -----------
-----
    clr tmp                          ; tmp = 0, удобно обнулять
переменные.
    sts min10, tmp                 ; Десятки минут = 0.
    sts min1, tmp                  ; Единицы минут = 0.
    sts sec10, tmp                 ; Десятки секунд = 0.
    sts sec1, tmp                  ; Единицы секунд = 0.
    sts  mux_pos,  tmp                                ;  Начинаем
мультиплексирование с разряда 0.
    sts tick_div, tmp                ; Делитель быстрых тиков
таймера = 0.
    sts btn0_state, tmp            ; Состояние кнопки PD0: 0 =
не нажата.
    sts  btn0_hold_l,  tmp              ; Младший байт времени
удержания PD0 = 0.
    sts  btn0_hold_h,  tmp              ; Старший байт времени
удержания PD0 = 0.
    sts btn0_repeat, tmp           ; Счётчик повторов PD0 = 0.
    sts btn1_state, tmp            ; Состояние кнопки PD1: 0 =
не нажата.
    sts  btn1_hold_l,  tmp              ; Младший байт времени
удержания PD1 = 0.
    sts  btn1_hold_h,  tmp              ; Старший байт времени
удержания PD1 = 0.
    sts btn1_repeat, tmp           ; Счётчик повторов PD1 = 0.
    sts run_flag, tmp                ; run_flag = 0: таймер не
запущен.
    sts finish_flag, tmp           ; finish_flag = 0: таймер ещё
не завершился.
    sts cd_ms_l, tmp                 ; Младший байт счётчика
миллисекунд обратного отсчёта.
    sts cd_ms_h, tmp                 ; Старший байт счётчика
миллисекунд обратного отсчёта.
    sts blink_ms_l, tmp              ; Младший байт счётчика
миллисекунд мигания.
```

```
    sts blink_ms_h, tmp                 ; Старший байт счётчика
миллисекунд мигания.
    sts blink_state, tmp             ; blink_state = 0: состояние
мигания "выключено".
    sts mode, tmp                   ; mode = 0: режим настройки.

    ; ---------------- Настройка внешних прерываний INT0/INT1 --
--------------
    ldi tmp, (1<<ISC01)|(1<<ISC00)|(1<<ISC11)|(1<<ISC10) ; INT0
и INT1 по фронту 0->1.
    out MCUCR, tmp                  ; Записываем настройку фронта
в MCUCR.
    ldi tmp, (1<<INT0)|(1<<INT1)        ; Разрешаем внешние
прерывания INT0 и INT1.
    out GICR, tmp                       ; Записываем разрешение в
регистр GICR.

    ; ---------------- Настройка Timer0 ----------------
    ; Timer0 работает в CTC: считает до OCR0 и вызывает
прерывание. OCR0 – регистр сравнения таймера
    ; Частота МК = 8 МГц, предделитель = 8.
    ; 8 МГц / 8 = 1 МГц, 1 тик = 1 мкс.
    ; OCR0 = 249, значит 250 тиков = 250 мкс = 0.25 мс.
    ldi tmp, 249                        ; Значение сравнения OCR0 =
249.
    out OCR0, tmp                   ; Записываем OCR0.
    ldi tmp, (1<<WGM01)|(1<<CS01)    ; WGM01=1: CTC, CS01=1:
предделитель 8.
    out TCCR0, tmp                          ; Запускаем Timer0 с
выбранным режимом.
    ldi tmp, (1<<OCIE0)             ; Разрешаем прерывание Timer0
Compare Match.
    out TIMSK, tmp                      ; Записываем разрешение в
TIMSK.
```

```
    sei                                 ; Глобально разрешаем все
прерывания.

MAIN:                                   ; Главный цикл программы.
    rjmp MAIN                           ; Ничего не делаем в основном
цикле, всё работает через прерывания.

;
=============================================================
=============
; INT0_ISR - обработчик кнопки PD2.
; PD2 запускает таймер, ставит на паузу или продолжает отсчёт.
;
=============================================================
=============
INT0_ISR:
    push tmp                            ; Сохраняем tmp, потому что в
прерывании будем его менять.
    push tmp2                           ; Сохраняем tmp2.
    push value                          ; Сохраняем value.
    in tmp, SREG                            ; Читаем SREG, где лежат
флаги процессора.
    push tmp                            ; Сохраняем SREG в стек.

    rcall IS_ZERO                       ; Проверяем, равно ли текущее
время 00.00.
    tst value                             ; Проверяем value: 1 значит
время равно нулю.
    brne int0_done                      ; Если время 00.00, запускать
нечего, выходим.

    ldi tmp, 1                          ; tmp = 1.
    sts mode, tmp                       ; Переводим программу в режим
работы.

    clr tmp                             ; tmp = 0.
```

```
    sts finish_flag, tmp            ; Сбрасываем флаг завершения.
    sts blink_state, tmp                ; Сбрасываем состояние
мигания.
    sts blink_ms_l, tmp                 ; Обнуляем младший байт
счётчика мигания.
    sts blink_ms_h, tmp                 ; Обнуляем старший байт
счётчика мигания.

    lds tmp, run_flag               ; Читаем run_flag: 0 = стоит,
1 = идёт.
    ldi tmp2, 1                     ; tmp2 = 1.
    eor tmp, tmp2                    ; Инвертируем младший бит: 0
станет 1, 1 станет 0.
    sts run_flag, tmp               ; Записываем новое состояние:
старт/пауза.

int0_done:
    pop tmp                         ; Достаём сохранённый SREG из
стека.
    out SREG, tmp                       ; Восстанавливаем флаги
процессора.
    pop value                       ; Восстанавливаем value.
    pop tmp2                        ; Восстанавливаем tmp2.
    pop tmp                         ; Восстанавливаем tmp.
    reti                            ; Возврат из прерывания.

;
=============================================================
=============
; INT1_ISR - обработчик кнопки PD3.
; PD3 переключает режим настройки и режим работы.
; mode = 0 -> настройка, mode = 1 -> работа.
;
=============================================================
=============
INT1_ISR:
```

```
    push tmp                        ; Сохраняем tmp.
    in tmp, SREG                    ; Читаем SREG.
    push tmp                        ; Сохраняем SREG.

    lds tmp, mode                   ; Читаем текущий режим.
    tst tmp                         ; Проверяем mode на ноль.
    brne int1_to_setup              ; Если mode != 0, значит были
в работе, идём в настройку.

    ldi tmp, 1                      ; tmp = 1.
    sts mode, tmp                   ; mode = 1: режим работы.
    clr tmp                         ; tmp = 0.
    sts run_flag, tmp               ; Таймер пока не запускаем.
    sts finish_flag, tmp            ; Сбрасываем флаг завершения.
    sts blink_state, tmp            ; Отключаем мигание.
    sts cd_ms_l, tmp                ; Обнуляем миллисекунды
обратного отсчёта.
    sts cd_ms_h, tmp                ; Обнуляем старший байт
миллисекунд.
    sts blink_ms_l, tmp             ; Обнуляем счётчик мигания.
    sts blink_ms_h, tmp             ; Обнуляем старший байт
счётчика мигания.
    rjmp int1_done                  ; Переходим к выходу из
прерывания.

int1_to_setup:
    clr tmp                         ; tmp = 0.
    sts mode, tmp                   ; mode = 0: режим настройки.
    sts run_flag, tmp               ; Останавливаем таймер.
    sts finish_flag, tmp            ; Сбрасываем завершение.
    sts blink_state, tmp            ; Отключаем мигание.
    sts cd_ms_l, tmp                ; Обнуляем счётчик
миллисекунд отсчёта.
    sts cd_ms_h, tmp                ; Обнуляем старший байт
счётчика.
    sts blink_ms_l, tmp             ; Обнуляем счётчик мигания.
```

```
    sts blink_ms_h, tmp                 ; Обнуляем старший байт
счётчика мигания.

int1_done:
    pop tmp                             ; Достаём сохранённый SREG.
    out SREG, tmp                       ; Восстанавливаем SREG.
    pop tmp                             ; Восстанавливаем tmp.
    reti                                ; Возврат из прерывания INT1.

;
=============================================================
=============
; INC_ONE_SECOND - увеличить установленное время на 1 секунду.
; Формат переменных: min10 min1 sec10 sec1.
; Например 01.23 хранится как min10=0, min1=1, sec10=2, sec1=3.
;
=============================================================
=============
INC_ONE_SECOND:
    lds tmp, sec1                       ; Загружаем единицы секунд.
    inc tmp                              ; Увеличиваем единицы секунд
на 1.
    cpi tmp, 10                         ; Сравниваем с 10.
    brlo save_sec1                          ; Если tmp < 10, просто
сохраняем sec1.
    clr tmp                              ; Если стало 10, сбрасываем
единицы секунд в 0.
    sts sec1, tmp                       ; Сохраняем sec1 = 0.

    lds tmp, sec10                      ; Загружаем десятки секунд.
    inc tmp                             ; Увеличиваем десятки секунд.
    cpi tmp, 6                           ; Десятки секунд могут быть
0..5, сравниваем с 6.
    brlo save_sec10                      ; Если tmp < 6, сохраняем
sec10.
```

```
    clr tmp                         ; Если стало 6, сбрасываем
десятки секунд в 0.
    sts sec10, tmp                  ; Сохраняем sec10 = 0.

    lds tmp, min1                   ; Загружаем единицы минут.
    inc tmp                         ; Увеличиваем единицы минут.
    cpi tmp, 10                     ; Проверяем переполнение 9 ->
10.
    brlo save_min1                  ; Если tmp < 10, сохраняем
min1.
    clr tmp                         ; Если стало 10, сбрасываем
единицы минут.
    sts min1, tmp                   ; Сохраняем min1 = 0.

    lds tmp, min10                  ; Загружаем десятки минут.
    inc tmp                         ; Увеличиваем десятки минут.
    cpi tmp, 10                     ; Проверяем переполнение
после 99.59.
    brlo save_min10                 ; Если tmp < 10, сохраняем
min10.
    clr tmp                         ; Если стало 10, возвращаемся
к 00.00.

save_min10:
    sts min10, tmp                  ; Сохраняем десятки минут.
    ret                             ; Возвращаемся из
подпрограммы.

save_min1:
    sts min1, tmp                   ; Сохраняем единицы минут.
    ret                             ; Возвращаемся.

save_sec10:
    sts sec10, tmp                  ; Сохраняем десятки секунд.
    ret                             ; Возвращаемся.
```

```
save_sec1:
    sts sec1, tmp                   ; Сохраняем единицы секунд.
    ret                             ; Возвращаемся.

;
=================================================================
=============
; DEC_ONE_SECOND - уменьшить время на 1 секунду.
; Если уже 00.00, ниже нуля не уходим.
;
=================================================================
=============
DEC_ONE_SECOND:
    lds tmp, min10                  ; Загружаем десятки минут.
    lds tmp2, min1                  ; Загружаем единицы минут.
    or tmp, tmp2                      ; Склеиваем через OR, чтобы
проверить, есть ли ненулевой разряд.
    lds tmp2, sec10                 ; Загружаем десятки секунд.
    or tmp, tmp2                          ; Добавляем к проверке
десятки секунд.
    lds tmp2, sec1                  ; Загружаем единицы секунд.
    or tmp, tmp2                          ; Добавляем к проверке
единицы секунд.
    breq dec_done                       ; Если все разряды нули,
ничего не уменьшаем.

    lds tmp, sec1                   ; Загружаем единицы секунд.
    tst tmp                         ; Проверяем, равны ли они 0.
    breq borrow_sec1                ; Если sec1 = 0, нужен заём.
    dec tmp                              ; Иначе просто уменьшаем
sec1.
    sts sec1, tmp                     ; Сохраняем новые единицы
секунд.
    ret                             ; Возвращаемся.

borrow_sec1:
```

```
    ldi tmp, 9                      ; При заёме единицы секунд
становятся 9.
    sts sec1, tmp                   ; Сохраняем sec1 = 9.

    lds tmp, sec10                  ; Загружаем десятки секунд.
    tst tmp                         ; Проверяем, равны ли они 0.
    breq borrow_sec10               ; Если sec10 = 0, нужен заём
из минут.
    dec tmp                         ; Иначе уменьшаем sec10.
    sts sec10, tmp                  ; Сохраняем десятки секунд.
    ret                             ; Возвращаемся.

borrow_sec10:
    ldi tmp, 5                      ; При заёме десятки секунд
становятся 5.
    sts sec10, tmp                  ; Сохраняем sec10 = 5.

    lds tmp, min1                   ; Загружаем единицы минут.
    tst tmp                         ; Проверяем, равны ли они 0.
    breq borrow_min1                ; Если min1 = 0, нужен заём
из десятков минут.
    dec tmp                         ; Иначе уменьшаем единицы
минут.
    sts min1, tmp                   ; Сохраняем min1.
    ret                             ; Возвращаемся.

borrow_min1:
    ldi tmp, 9                      ; При заёме единицы минут
становятся 9.
    sts min1, tmp                   ; Сохраняем min1 = 9.

    lds tmp, min10                  ; Загружаем десятки минут.
    dec tmp                         ; Уменьшаем десятки минут.
    sts min10, tmp                  ; Сохраняем десятки минут.

dec_done:
```

```
    ret                                         ; Возвращаемся из
DEC_ONE_SECOND.

;
============================================================
=============
; IS_ZERO - проверить, равно ли время 00.00.
; Выход:
;   value = 1, если время 00.00;
;   value = 0, если время не 00.00.
;
============================================================
=============
IS_ZERO:
    clr value                        ; По умолчанию считаем, что
время не ноль.
    lds tmp, min10                  ; Читаем десятки минут.
    lds tmp2, min1                  ; Читаем единицы минут.
    or tmp, tmp2                        ; OR: если хотя бы один
разряд не 0, результат не 0.
    lds tmp2, sec10                 ; Читаем десятки секунд.
    or tmp, tmp2                    ; Добавляем их к проверке.
    lds tmp2, sec1                  ; Читаем единицы секунд.
    or tmp, tmp2                    ; Добавляем их к проверке.
    brne is_zero_ret                ; Если результат не 0, значит
время не 00.00.
    ldi value, 1                     ; Если все разряды 0, ставим
value = 1.
is_zero_ret:
    ret                             ; Возвращаемся.

;
============================================================
=============
; TIMER0_COMP_ISR - обработчик Timer0.
; Вызывается каждые 0.25 мс.
```

```
; Каждый вызов обновляет один разряд индикатора.
; Каждые 4 вызова выполняются действия с шагом 1 мс:
;   кнопки, обратный отсчёт, мигание.
;
================================================================
=============
TIMER0_COMP_ISR:
    push tmp                        ; Сохраняем tmp.
    push tmp2                       ; Сохраняем tmp2.
    push digit                      ; Сохраняем digit.
    push value                      ; Сохраняем value.
    push pins                       ; Сохраняем pins.
    push ZL                             ; Сохраняем младший байт
регистра Z.
    push ZH                             ; Сохраняем старший байт
регистра Z.
    in tmp, SREG                    ; Читаем флаги процессора.
    push tmp                        ; Сохраняем SREG.

    clr tmp                         ; tmp = 0.
    out PORTA, tmp                   ; Выключаем все разряды перед
сменой сегментов.

    lds digit, mux_pos                  ; digit = номер разряда,
который сейчас надо вывести.

    lds tmp, finish_flag                ; Читаем флаг завершения
таймера.
    tst tmp                         ; Проверяем finish_flag.
    breq normal_display               ; Если таймер не завершён,
показываем обычное значение.
    lds tmp, blink_state                ; Если завершён, читаем
состояние мигания.
    tst tmp                         ; Проверяем blink_state.
    breq display_blank               ; Если blink_state = 0, гасим
индикатор.
```

```
normal_display:
    ldi ZL, low(min10)              ; ZL = младший байт адреса
первого разряда min10.
    ldi ZH, high(min10)             ; ZH = старший байт адреса
первого разряда min10.
    add ZL, digit                  ; Прибавляем номер разряда к
адресу.
    adc ZH, zero                   ; Добавляем перенос в старший
байт адреса.
    ld value, Z                       ; value = цифра текущего
разряда.

    ldi ZL, low(SEG_TABLE*2)        ; ZL = младший байт адреса
таблицы сегментов во Flash.
    ldi ZH, high(SEG_TABLE*2)       ; ZH = старший байт адреса
таблицы сегментов.
    add ZL, value                     ; Прибавляем цифру 0..9 к
адресу таблицы.
    adc ZH, zero                   ; Добавляем перенос в старший
байт.
    lpm tmp, Z                        ; Загружаем из Flash код
сегментов для цифры.

    cpi digit, 1                      ; Проверяем, выводится ли
второй разряд.
    brne no_dot                    ; Если это не второй разряд,
точку не включаем.
    ori tmp, 0x80                         ; Если второй разряд,
включаем DP для формата MM.SS.
no_dot:
    out PORTC, tmp                    ; Выводим код сегментов на
PORTC.
    rjmp select_digit              ; Переходим к выбору нужного
разряда.
```

```
display_blank:
    clr tmp                          ; Для активной единицы 0x00
гасит все сегменты.
    out PORTC, tmp                  ; Гасим сегменты.

select_digit:
    ldi tmp, 0x08                       ; На новой плате разряды
подключены наоборот: начинаем с PA3.
select_loop:
    tst digit                        ; Проверяем, сколько раз надо
сдвинуть маску.
    breq select_done                    ; Если digit = 0, нужная
маска уже готова.
    lsr tmp                          ; Сдвигаем 1 вправо: PA3 ->
PA2 -> PA1 -> PA0.
    dec digit                       ; Уменьшаем счётчик сдвигов.
    rjmp select_loop                    ; Повторяем, пока digit не
станет 0.
select_done:
    out PORTA, tmp                      ; Включаем нужный разряд
индикатора.

    lds tmp, mux_pos                    ; Читаем текущую позицию
мультиплексирования.
    inc tmp                             ; Переходим к следующему
разряду.
    cpi tmp, 4                       ; Проверяем, не прошли ли 4
разряда.
    brlo save_mux                   ; Если tmp < 4, сохраняем как
есть.
    clr tmp                         ; Если было 4, возвращаемся к
разряду 0.
save_mux:
    sts mux_pos, tmp                    ; Сохраняем новую позицию
мультиплексирования.
```

```
    lds tmp, tick_div                ; Читаем делитель быстрых
тиков.
    inc tmp                         ; Увеличиваем его на 1.
    cpi tmp, 4                      ; 4 * 0.25 мс = 1 мс.
    brlo save_tick_div               ; Если ещё не набрали 1 мс,
только сохраняем делитель.
    clr tmp                                ; Если набрали 1 мс,
сбрасываем делитель.
    sts tick_div, tmp               ; Сохраняем tick_div = 0.

    rcall BUTTONS_TICK              ; Обрабатываем кнопки PD0/PD1
раз в 1 мс.
    rcall COUNTDOWN_TICK                ; Обрабатываем обратный
отсчёт раз в 1 мс.
    rcall BLINK_TICK                ; Обрабатываем мигание раз в
1 мс.
    rjmp timer0_exit                        ; Идём к выходу из
прерывания.

save_tick_div:
    sts tick_div, tmp               ; Сохраняем tick_div, если 1
мс ещё не прошла.

timer0_exit:
    pop tmp                         ; Достаём сохранённый SREG.
    out SREG, tmp                   ; Восстанавливаем SREG.
    pop ZH                          ; Восстанавливаем ZH.
    pop ZL                          ; Восстанавливаем ZL.
    pop pins                        ; Восстанавливаем pins.
    pop value                       ; Восстанавливаем value.
    pop digit                       ; Восстанавливаем digit.
    pop tmp2                        ; Восстанавливаем tmp2.
    pop tmp                         ; Восстанавливаем tmp.
    reti                                 ; Возврат из прерывания
Timer0.
```

```
;
================================================================
=============
; COUNTDOWN_TICK - логика обратного отсчёта.
; Вызывается раз в 1 мс.
; Если run_flag = 1, считает 1000 мс и уменьшает время на 1
секунду.
;
================================================================
=============
COUNTDOWN_TICK:
    lds tmp, run_flag              ; Читаем run_flag.
    tst tmp                              ; Проверяем, запущен ли
таймер.
    breq countdown_ret               ; Если run_flag = 0, ничего
не делаем.

    lds tmp, cd_ms_l                   ; Загружаем младший байт
счётчика миллисекунд.
    lds tmp2, cd_ms_h                  ; Загружаем старший байт
счётчика миллисекунд.
    subi tmp, low(-1)                ; Увеличиваем 16-битное число
на 1: младший байт.
    sbci tmp2, high(-1)              ; Увеличиваем 16-битное число
на 1: старший байт с переносом.

    cpi tmp, low(1000)                ; Сравниваем младший байт с
low(1000).
    ldi digit, high(1000)              ; Загружаем high(1000) во
временный регистр.
    cpc tmp2, digit                   ; Сравниваем старший байт с
high(1000) с учётом флагов.
    brlo countdown_save_ms           ; Если прошло меньше 1000 мс,
только сохраняем счётчик.
```

```
    rcall DEC_ONE_SECOND            ; Прошла 1 секунда, уменьшаем
время на 1 секунду.
    rcall IS_ZERO                   ; Проверяем, стало ли 00.00.
    tst value                           ; value = 1, если время
00.00.
    breq countdown_clear_ms          ; Если ещё не ноль, просто
обнуляем счётчик миллисекунд.

    clr value                       ; value = 0.
    sts run_flag, value             ; Останавливаем таймер.
    ldi value, 1                    ; value = 1.
    sts finish_flag, value          ; Ставим флаг завершения.
    sts blink_state, value          ; Начинаем с состояния "00.00
горит".
    clr value                       ; value = 0.
    sts blink_ms_l, value           ; Обнуляем счётчик мигания.
    sts  blink_ms_h,  value             ; Обнуляем старший байт
счётчика мигания.

countdown_clear_ms:
    clr tmp                         ; tmp = 0.
    clr tmp2                        ; tmp2 = 0.

countdown_save_ms:
    sts  cd_ms_l,  tmp                  ; Сохраняем младший байт
миллисекунд.
    sts  cd_ms_h,  tmp2                 ; Сохраняем старший байт
миллисекунд.

countdown_ret:
    ret                             ; Возвращаемся.

;
=============================================================
=============
; BLINK_TICK - мигание 00.00 при завершении.
```

```
; Вызывается раз в 1 мс.
; Меняет blink_state каждые 250 мс:
;    250 мс горит, 250 мс не горит -> полный период 500 мс -> 2 Гц.
;
================================================================
=============
BLINK_TICK:
    lds tmp, finish_flag           ; Читаем флаг завершения.
    tst tmp                        ; Проверяем finish_flag.
    breq blink_ret                  ; Если таймер не завершён, не мигаем.

    lds  tmp,  blink_ms_l               ; Загружаем младший байт счётчика мигания.
    lds  tmp2,  blink_ms_h              ; Загружаем старший байт счётчика мигания.
    subi tmp, low(-1)              ; Увеличиваем счётчик мигания на 1 мс.
    sbci tmp2, high(-1)             ; Увеличиваем старший байт с переносом.

    cpi tmp, low(250)                ; Сравниваем младший байт с low(250).
    ldi digit, high(250)           ; Загружаем high(250).
    cpc tmp2, digit                ; Сравниваем старший байт.
    brlo blink_save                 ; Если меньше 250 мс, только сохраняем счётчик.

    clr tmp                                ; Если прошло 250 мс, обнуляем младший байт.
    clr tmp2                       ; Обнуляем старший байт.
    lds value, blink_state           ; Читаем текущее состояние мигания.
    ldi digit, 1                   ; digit = 1.
```

```
    eor value, digit                ; Инвертируем blink_state: 0-
>1, 1->0.
    sts blink_state, value          ; Сохраняем новое состояние
мигания.

blink_save:
    sts blink_ms_l, tmp             ; Сохраняем младший байт
счётчика мигания.
    sts blink_ms_h, tmp2            ; Сохраняем старший байт
счётчика мигания.

blink_ret:
    ret                             ; Возвращаемся.

;
=================================================================
=============
; BUTTONS_TICK - обработка кнопок PD0 и PD1.
; Вызывается раз в 1 мс.
;  PD0/PD1 работают только в режиме настройки и только если
таймер не запущен.
;
=================================================================
=============
BUTTONS_TICK:
    lds tmp, mode                   ; Читаем режим.
    tst tmp                         ; mode = 0 значит настройка.
    brne buttons_ret                ; Если mode != 0, кнопки
PD0/PD1 не меняют значение.

    lds tmp, run_flag               ; Читаем run_flag.
    tst tmp                         ; Проверяем, не идёт ли
обратный отсчёт.
    brne buttons_ret                ; Если таймер идёт, PD0/PD1
не работают.
```

```
    lds tmp, finish_flag          ; Читаем finish_flag.
    tst tmp                       ; Проверяем, не завершился ли
таймер.
    brne buttons_ret              ; Если мигает 00.00, PD0/PD1
не работают.

    in pins, PIND                 ; Читаем состояние кнопок на
PORTD.

    sbrc pins, 0                  ; Если bit0 = 1, значит PD0
нажата.
    rcall BTN0_PRESSED            ; Обрабатываем нажатую PD0.
    sbrs pins, 0                  ; Если bit0 = 0, значит PD0
отпущена.
    rcall BTN0_RELEASED           ; Обрабатываем отпускание
PD0.

    sbrc pins, 1                  ; Если bit1 = 1, значит PD1
нажата.
    rcall BTN1_PRESSED            ; Обрабатываем нажатую PD1.
    sbrs pins, 1                  ; Если bit1 = 0, значит PD1
отпущена.
    rcall BTN1_RELEASED           ; Обрабатываем отпускание
PD1.
buttons_ret:
    ret                           ; Возвращаемся.

BTN0_RELEASED:
    clr tmp                       ; tmp = 0.
    sts btn0_state, tmp           ; Запоминаем, что PD0
отпущена.
    sts btn0_hold_l, tmp          ; Обнуляем младший байт
времени удержания PD0.
    sts btn0_hold_h, tmp          ; Обнуляем старший байт
времени удержания PD0.
```

```
    sts btn0_repeat, tmp            ; Обнуляем счётчик повторов
PD0.
    ret                             ; Возвращаемся.

BTN1_RELEASED:
    clr tmp                         ; tmp = 0.
    sts btn1_state, tmp                   ; Запоминаем, что PD1
отпущена.
    sts btn1_hold_l, tmp                ; Обнуляем младший байт
времени удержания PD1.
    sts btn1_hold_h, tmp                ; Обнуляем старший байт
времени удержания PD1.
    sts btn1_repeat, tmp            ; Обнуляем счётчик повторов
PD1.
    ret                             ; Возвращаемся.

BTN0_PRESSED:
    lds tmp, btn0_state              ; Читаем прошлое состояние
PD0.
    tst tmp                         ; Проверяем: была ли кнопка
уже нажата.
    brne btn0_hold                  ; Если уже была нажата, это
удержание.

    ldi tmp, 1                      ; tmp = 1.
    sts btn0_state, tmp             ; Запоминаем, что PD0 нажата.
    clr tmp                         ; tmp = 0.
    sts btn0_hold_l, tmp             ; Обнуляем время удержания
PD0.
    sts btn0_hold_h, tmp                ; Обнуляем старший байт
времени удержания.
    sts btn0_repeat, tmp            ; Обнуляем счётчик повторов.
    rcall  INC_ONE_SECOND                 ; Первое действие при
нажатии: +1 секунда сразу.
    ret                             ; Возвращаемся.
```

```
btn0_hold:
    lds tmp, btn0_hold_l              ; Загружаем младший байт
времени удержания PD0.
    lds tmp2, btn0_hold_h             ; Загружаем старший байт
времени удержания PD0.
    subi tmp, low(-1)                ; Увеличиваем время удержания
на 1 мс.
    sbci tmp2, high(-1)               ; Увеличиваем старший байт с
переносом.
    sts btn0_hold_l, tmp              ; Сохраняем младший байт
времени удержания.
    sts btn0_hold_h, tmp2             ; Сохраняем старший байт
времени удержания.

    cpi tmp, low(2000)                ; Проверяем момент ровно 2000
мс.
    ldi digit, high(2000)            ; Загружаем high(2000).
    cpc tmp2, digit                  ; Сравниваем старший байт.
    breq btn0_repeat_now              ; На отметке 2 с делаем
повтор сразу.

    cpi tmp, low(4000)                ; Проверяем момент ровно 4000
мс.
    ldi digit, high(4000)            ; Загружаем high(4000).
    cpc tmp2, digit                  ; Сравниваем старший байт.
    breq btn0_repeat_now              ; На отметке 4 с делаем
повтор сразу.

    rcall GET_REPEAT_PERIOD           ; Получаем период повтора: 0,
200 или 100 мс.
    tst value                        ; Проверяем value.
    breq btn0_done                    ; Если value = 0, удержание
ещё меньше 2 с.

    lds tmp, btn0_repeat              ; Читаем счётчик повторов
PD0.
```

```
    inc tmp                          ; Увеличиваем счётчик
повторов на 1 мс.
    cp tmp, value                    ; Сравниваем с нужным
периодом.
    brlo btn0_save_repeat            ; Если период ещё не набран,
просто сохраняем.
btn0_repeat_now:
    clr tmp                          ; Сбрасываем счётчик
повторов.
    rcall INC_ONE_SECOND             ; Выполняем повторное +1
секунда.

btn0_save_repeat:
    sts btn0_repeat, tmp             ; Сохраняем счётчик повторов
PD0.
btn0_done:
    ret                              ; Возвращаемся.

BTN1_PRESSED:
    lds tmp, btn1_state              ; Читаем прошлое состояние
PD1.
    tst tmp                          ; Проверяем: была ли кнопка
уже нажата.
    brne btn1_hold                   ; Если уже была нажата, это
удержание.

    ldi tmp, 1                       ; tmp = 1.
    sts btn1_state, tmp              ; Запоминаем, что PD1 нажата.
    clr tmp                          ; tmp = 0.
    sts btn1_hold_l, tmp             ; Обнуляем время удержания
PD1.
    sts btn1_hold_h, tmp             ; Обнуляем старший байт
времени удержания.
    sts btn1_repeat, tmp             ; Обнуляем счётчик повторов.
    rcall DEC_ONE_SECOND             ; Первое действие при
нажатии: -1 секунда сразу.
```

```
    ret                             ; Возвращаемся.

btn1_hold:
    lds tmp, btn1_hold_l            ; Загружаем младший байт
времени удержания PD1.
    lds tmp2, btn1_hold_h           ; Загружаем старший байт
времени удержания PD1.
    subi tmp, low(-1)               ; Увеличиваем время удержания
на 1 мс.
    sbci tmp2, high(-1)             ; Увеличиваем старший байт с
переносом.
    sts btn1_hold_l, tmp            ; Сохраняем младший байт
времени удержания.
    sts btn1_hold_h, tmp2           ; Сохраняем старший байт
времени удержания.

    cpi tmp, low(2000)              ; Проверяем момент ровно 2000
мс.
    ldi digit, high(2000)           ; Загружаем high(2000).
    cpc tmp2, digit                 ; Сравниваем старший байт.
    breq btn1_repeat_now            ; На отметке 2 с делаем
повтор сразу.

    cpi tmp, low(4000)              ; Проверяем момент ровно 4000
мс.
    ldi digit, high(4000)           ; Загружаем high(4000).
    cpc tmp2, digit                 ; Сравниваем старший байт.
    breq btn1_repeat_now            ; На отметке 4 с делаем
повтор сразу.

    rcall GET_REPEAT_PERIOD         ; Получаем период повтора: 0,
200 или 100 мс.
    tst value                       ; Проверяем value.
    breq btn1_done                  ; Если value = 0, удержание
ещё меньше 2 с.
```

```
    lds tmp, btn1_repeat                ; Читаем счётчик повторов
PD1.
    inc tmp                             ; Увеличиваем счётчик
повторов на 1 мс.
    cp tmp, value                       ; Сравниваем с нужным
периодом.
    brlo btn1_save_repeat               ; Если период ещё не набран,
просто сохраняем.
btn1_repeat_now:
    clr tmp                             ; Сбрасываем счётчик
повторов.
    rcall DEC_ONE_SECOND                ; Выполняем повторное -1
секунда.

btn1_save_repeat:
    sts btn1_repeat, tmp                ; Сохраняем счётчик повторов
PD1.
btn1_done:
    ret                                 ; Возвращаемся.

;
================================================================
=============
; GET_REPEAT_PERIOD - определить период автоповтора кнопки.
; Вход:
;   tmp2:tmp = время удержания кнопки в миллисекундах.
; Выход:
;   value = 0, если меньше 2 секунд;
;   value = 200, если от 2 до 4 секунд;
;   value = 100, если больше 4 секунд.
;
================================================================
=============
GET_REPEAT_PERIOD:
    clr value                           ; По умолчанию автоповтора
нет.
```

```
    cpi tmp, low(2000)                ; Сравниваем младший байт
удержания с low(2000).
    ldi digit, high(2000)           ; Загружаем high(2000).
    cpc tmp2, digit                   ; Сравниваем старший байт
удержания.
    brlo repeat_done                ; Если меньше 2000 мс, value
остаётся 0.

    ldi value, 200                    ; После 2 секунд повтор
каждые 200 мс.

    cpi tmp, low(4000)                ; Сравниваем младший байт
удержания с low(4000).
    ldi digit, high(4000)           ; Загружаем high(4000).
    cpc tmp2, digit                   ; Сравниваем старший байт
удержания.
    brlo repeat_done                  ; Если меньше 4000 мс,
оставляем value = 200.

    ldi value, 100                    ; После 4 секунд повтор
каждые 100 мс.

repeat_done:
    ret                             ; Возвращаемся.

;
=============================================================
=============
;       SEG_TABLE  -  таблица  кодов  цифр  для  семисегментного
индикатора.
;   Биты соответствуют:
;     bit0=A,  bit1=B,  bit2=C,  bit3=D,  bit4=E,  bit5=F,  bit6=G,
bit7=DP.
```

```
;
================================================================
=============
SEG_TABLE:
    .db 0x3F, 0x06, 0x5B, 0x4F, 0x66, 0x6D, 0x7D, 0x07, 0x7F,
0x6F
                                        ;          Коды          цифр
0,1,2,3,4,5,6,7,8,9.

.dseg                                  ; Далее сегмент данных SRAM.
min10:      .byte 1                    ; Десятки минут.
min1:       .byte 1                    ; Единицы минут.
sec10:      .byte 1                    ; Десятки секунд.
sec1:       .byte 1                    ; Единицы секунд.
mux_pos:    .byte 1                     ; Номер текущего разряда для
мультиплексирования.
tick_div:   .byte 1                    ; Делитель 0.25 мс тиков до 1
мс.
btn0_state: .byte 1                     ; Состояние PD0: 0 отпущена,
1 нажата.
btn0_hold_l:.byte 1                          ; Младший байт времени
удержания PD0.
btn0_hold_h:.byte 1                          ; Старший байт времени
удержания PD0.
btn0_repeat:.byte 1                    ; Счётчик периода автоповтора
PD0.
btn1_state: .byte 1                     ; Состояние PD1: 0 отпущена,
1 нажата.
btn1_hold_l:.byte 1                          ; Младший байт времени
удержания PD1.
btn1_hold_h:.byte 1                          ; Старший байт времени
удержания PD1.
btn1_repeat:.byte 1                    ; Счётчик периода автоповтора
PD1.
run_flag:   .byte 1                      ; 0 таймер стоит, 1 таймер
идёт.
```

```
finish_flag:.byte 1                    ; 1 значит таймер дошёл до
00.00.
cd_ms_l:     .byte 1                   ; Младший байт миллисекунд
обратного отсчёта.
cd_ms_h:     .byte 1                   ; Старший байт миллисекунд
обратного отсчёта.
blink_ms_l: .byte 1                    ; Младший байт миллисекунд
мигания.
blink_ms_h: .byte 1                    ; Старший байт миллисекунд
мигания.
blink_state:.byte 1                     ; 0 индикатор погашен, 1
00.00 отображается.
mode:        .byte 1                  ; 0 режим настройки, 1 режим
работы.
```